Deutsche Bank - AVP Java Developer Interview Questionnaire (With Detailed Answers)

Java (Core & Java 17 Features)

Explain the use of records in Java 17. How are they different from normal classes?

Records provide a concise way to declare classes that are transparent carriers for immutable data. A record automatically generates a constructor, accessors (methods named after components), equals(), hashCode(), and toString(), reducing boilerplate. Records are implicitly final and their fields are private and final. Use records when you want a simple immutable data holder. Avoid records when you need mutable state, complex validation in setters, or when you must extend the record (records cannot be extended).

```
public record Person(String name, int age) {}

// Generated methods: Person(String name, int age), name(), age(), equals(), hashCode(), toString()
```

What are sealed classes and when would you use them?

Sealed classes restrict which other classes or interfaces may extend or implement them. You declare a sealed class with the 'sealed' modifier and list permitted subclasses using the 'permits' clause. This enables exhaustive pattern matching, safer hierarchies, and better control over APIs. Use sealed classes when you want a closed set of implementations (e.g., algebraic data types) and want to prevent external implementations.

```
public sealed interface Shape permits Circle, Rectangle, Triangle {}
public final class Circle implements Shape { ... }
public final class Rectangle implements Shape { ... }
```

How does pattern matching for instanceof improve readability?

Pattern matching allows you to both test and cast in a single concise expression. Previously you had to write an instanceof test and then cast: if (obj instanceof String) { String s = (String) obj; ... } With pattern matching: if (obj instanceof String s) { ... } This reduces boilerplate and eliminates the risk of casting errors between the check and cast.

```
Object o = "hello";
if (o instanceof String s) {
    System.out.println(s.toUpperCase());
}
```

Explain switch expressions with an example.

Switch expressions unify switch statements and provide a value-returning form plus a simplified arrow-label syntax. They make code less error-prone by avoiding fall-through and by enabling the compiler to check exhaustiveness (especially with enums or sealed types). Use 'yield' to return values from complex branches.

```
int day = 3;
String name = switch (day) {
  case 1 -> "Mon";
  case 2 -> "Tue";
  case 3 -> "Wed";
  default -> "Unknown";
};
```

What are text blocks and how do they help with multi-line strings?

Text blocks (triple-quoted style) make multi-line string literals easier to read and maintain by preserving natural formatting and reducing escape noise. They're ideal for SQL, JSON, XML, or HTML snippets embedded in code. They also allow easier control over indentation and avoid many concatenations.

```
String json = """
{
  "name": "Alice",
```

```
  "age": 30
}
""";
```

How does Java handle memory management and garbage collection?

Java separates memory into stack (thread-local frames and primitives/references) and heap (objects). The JVM's garbage collector (GC) automatically reclaims unreachable objects. Modern JVMs use generational GC: young generation for short-lived objects and old generation for long-lived objects. G1 is the common default collector—it focuses on pause-time predictability and concurrent marking. There are also low-latency collectors (ZGC, Shenandoah) for specific use cases. Understanding GC logs, heap sizing, and object allocation patterns is critical for performance tuning.

Explain the difference between parallel streams and CompletableFuture.

Parallel streams are a high-level API for data-parallel operations using the ForkJoinPool.commonPool by default. They're convenient for CPU-bound bulk operations but less flexible for orchestration. CompletableFuture is a composable asynchronous API that supports explicit control over threads (via Executors), non-blocking composition (thenApplyAsync, thenCompose), and more sophisticated error handling and timeouts. Use parallel streams for simple, element-wise parallelism; use CompletableFuture for complex async workflows and I/O-bound tasks.

What are common pitfalls in multithreading in Java?

Common pitfalls include race conditions (shared mutable state without synchronization), deadlocks (cyclic locking order), livelocks, visibility issues (missing volatile or synchronization leading to stale reads), incorrect use of wait/notify, and improper thread lifecycle management. Prefer higher-level concurrency utilities (Executors, ConcurrentHashMap, CountDownLatch, Semaphore) over raw Thread and synchronized wherever possible. Also watch for blocking calls on limited thread pools and thread leakage.

Spring Boot & Microservices

Explain how auto-configuration works in Spring Boot.

Auto-configuration uses classpath scanning and conditional annotations to create beans automatically when certain classes or properties are present. Spring Boot leverages 'spring.factories' (or newer 'spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports' mechanism) to register auto-configuration classes. Each auto-configuration class uses @ConditionalOnClass, @ConditionalOnProperty, etc., to only apply when appropriate. This reduces boilerplate but you should understand what is being auto-configured and override defaults when necessary.

What are profiles in Spring Boot and why are they used?

Profiles (spring.profiles) let you partition configuration and beans by environment or role (e.g., 'dev', 'qa', 'prod'). You can annotate beans with @Profile or keep environment-specific properties in application-dev.yml, application-prod.yml. Profiles are used to avoid hardcoding environment differences and to make deployments reproducible.

How do you secure REST APIs using JWT or OAuth2 in Spring Boot?

JWT: issue a signed token after authentication containing claims (user id, roles). Clients include the token in Authorization header (Bearer). On server, validate signature and claims, then map to GrantedAuthorities. OAuth2: use an Authorization Server (or identity provider) to handle flows (authorization code, client credentials). Spring Security provides resource-server support to validate tokens and integrate with OAuth2 providers. Key considerations: token expiry, refresh tokens, token revocation strategies, scoping of claims, and transport-level security (HTTPS).

Explain circuit breaker and retries in microservices.

A circuit breaker prevents cascading failures by stopping calls to a failing downstream service after repeated failures and switching to a fallback behavior. It has states: CLOSED -> OPEN -> HALF_OPEN -> CLOSED. Retries allow transient failures to be retried with configurable backoff, but careless retries can amplify load. Use libraries like Resilience4j to configure circuit-breakers, rate limiters, bulkheads, and retries, and combine them thoughtfully.

How would you manage configuration across multiple environments?

Use externalized configuration: Spring Cloud Config Server, Consul, or environment variables/ConfigMaps for Kubernetes/OpenShift. Keep secrets in Vault or a secret manager. Use property placeholders and profile-specific files. Maintain a clear promotion path (e.g., dev->qa->staging->prod) and ensure immutable configuration where possible.

What is the difference between @Component, @Service, and @Repository?

@Component is a generic stereotype for spring-managed components. @Service indicates a service-layer component (business logic) and is mainly for semantic clarity. @Repository is a persistence-layer stereotype and also activates Spring's exception translation for persistence exceptions into DataAccessException (a consistent unchecked hierarchy). Functionally they are similar but provide semantic clarity and slight framework behaviors.

How do you ensure backward compatibility when versioning APIs?

Version APIs via URI (e.g., /v1/payments), headers, or media types. For compatibility: avoid removing fields (add new optional fields instead), keep old behavior for defaults, provide deprecation warnings, and maintain clear migration guides. Use feature toggles for gradual rollouts and contract testing (e.g., Pact) to ensure clients aren’t broken.

Databases (Oracle)

Explain the difference between INNER JOIN, LEFT JOIN, and EXISTS.

INNER JOIN returns rows where join condition matches in both tables. LEFT JOIN returns all rows from the left table and matched rows from the right (NULL where no match). EXISTS checks for the existence of matching rows in a subquery and can be more efficient than IN for correlated subqueries because it can short-circuit; it’s often preferred when you want to test existence rather than return columns.

```
SELECT a.* FROM A a INNER JOIN B b ON a.id = b.a_id;
SELECT a.*, b.col FROM A a LEFT JOIN B b ON a.id = b.a_id;
-- Using EXISTS
SELECT a.* FROM A a WHERE EXISTS (SELECT 1 FROM B b WHERE b.a_id = a.id);
```

How do you analyze and improve a slow SQL query?

Steps: capture the query and use EXPLAIN PLAN to see the execution strategy; check indexes used and statistics; look for full table scans, large sorts, or hash joins; examine predicate selectivity and remove functions on indexed columns; rewrite queries (use EXISTS vs IN, avoid correlated subqueries), add or adjust indexes, partition large tables, and consider materialized views for expensive aggregations. Also review network latency and fetch sizes for JDBC.

What are indexes and when should they be avoided?

Indexes speed up reads by allowing the DB to find rows without scanning the entire table. They consume disk and memory, slow down writes (UPDATE/INSERT/DELETE), and may be inefficient for low-cardinality columns (e.g., boolean flags). Avoid redundant indexes, overly wide composite indexes, and indexing columns mainly used in ORDER BY if queries do not select/filter by them. Monitor index usage and remove unused ones.

Explain the difference between optimistic and pessimistic locking.

Optimistic locking assumes conflicts are rare: use a version column (or timestamp) and check the version on update, failing if changed (application retries). Pessimistic locking uses DB locks (SELECT ... FOR UPDATE) to prevent concurrent modifications by blocking other transactions until commit—useful when conflicts are frequent or when you must guarantee sequential access.

What are different transaction isolation levels?

Common isolation levels: READ UNCOMMITTED (dirty reads allowed), READ COMMITTED (prevents dirty reads), REPEATABLE READ (prevents non-repeatable reads), and SERIALIZABLE (strictest—prevents phantom reads). Oracle’s default is READ COMMITTED; choose levels based on correctness needs vs concurrency. Higher isolation reduces anomalies but can increase locking and reduce throughput.

How do you perform batch operations efficiently in Oracle?

Use JDBC batch operations (addBatch/executeBatch) with an appropriate commit frequency to reduce network round-trips. Use array binding (binding arrays of values to a PL/SQL block) for large inserts from applications. Use bulk operations in PL/SQL (FORALL, BULK COLLECT) for server-side efficiency. Tune commit intervals to balance rollback segment size and memory consumption.

Messaging (Kafka)

Explain how Kafka ensures fault tolerance and durability.

Kafka stores data in partitions replicated across multiple brokers. Each partition has a leader (handles reads/writes) and followers that replicate the data. The replication factor determines durability; producers can configure acks (0, 1, all) to control durability guarantees. Kafka's append-only log and segment-based storage make it resilient; brokers can fail and new leaders are elected from In-Sync Replicas (ISRs) to preserve availability and durability.

What are partitions and consumer groups in Kafka?

Partitions divide a topic's data into ordered logs enabling parallelism. A consumer group allows multiple consumers to coordinate consumption: each partition is consumed by at most one consumer in a group, enabling horizontal scaling. Ordering is guaranteed within a partition but not across partitions.

How do you achieve message ordering in Kafka?

To preserve ordering you must write related messages to the same partition—usually achieved by using a message key and a partitioner. If total ordering is required across all messages, use a single partition (which limits throughput). For per-entity ordering, use the entity identifier as key.

Explain exactly-once semantics in Kafka.

Exactly-once semantics (EOS) in Kafka involves idempotent producers and transactional writes across producers and consumers. Producers can enable idempotence to avoid duplicate appends, and Kafka transactions allow a consumer to read, process, and write offset commits atomically with outgoing messages—preventing duplicates during failures when configured correctly. EOS has operational complexity and performance costs; often idempotency or deduplication at the consumer is simpler.

What are common challenges when handling retries in Kafka consumers?

Challenges include poison-pill messages that always fail, ordering disruption if you move a message to a retry topic, blowup of message backlog, and duplicate processing. Common patterns: exponential backoff with delayed/retry topics, dead-letter queues for failed messages after retries, and idempotent processing to tolerate duplicates. Ensure transactional boundaries and offset commits are handled correctly.

Caching (Redis)

What are common cache eviction policies in Redis?

Redis supports several eviction policies: noeviction (error on writes), allkeys-lru (evict least-recently-used across all keys), volatile-lru (LRU on keys with TTL), allkeys-lfu/volatile-lfu (least-frequently-used), and volatile-ttl (evict keys with shortest TTL). Choose based on access patterns and whether TTLs are used.

How do you handle cache consistency with the database?

Common strategies: cache-aside (application reads DB on cache miss and populates cache; updates remove/invalidate cache), write-through (writes go through cache and persist to DB synchronously), and write-behind (async writes to DB). Cache invalidation is notoriously hard—prefer simple, well-tested invalidation and strong monitoring. Use versioning or TTLs for soft consistency guarantees.

Explain cache stampede and how to prevent it.

Cache stampede happens when many clients simultaneously miss the cache for a hot key and overwhelm the DB. Prevent by using locking/coalescing (single flight), request queuing, early refresh (refresh before expiry), probabilistic early expiration, or serving slightly stale values while refreshing in background. Redis-based locks (SET NX PX) or single-flight libraries are common solutions.

What are distributed locks in Redis and when would you use them?

Distributed locks coordinate access across processes. Implement using SET key value NX PX timeout (atomic set-if-not-exists with expiry) or use Redisson/Redlock libraries for higher-level semantics. Use locks sparingly (they reduce concurrency) and design for eventual consistency; ensure locks have TTLs to prevent deadlocks caused by crashed holders. Be aware of edge cases in network partitions—design for safety and be conservative.

DevOps / Deployment

Explain how Jenkins pipelines are structured.

Jenkins pipelines come in two styles: Declarative and Scripted. Declarative pipelines provide a structured syntax with stages, steps, and agents and are easier to read and maintain. Pipelines are defined in Jenkinsfile and can perform checkout, build, test, package, and deploy steps. Reusable pipeline libraries (shared libraries) help standardize builds across projects. Keep pipeline stages atomic and fast; parallelize independent tasks.

How would you implement CI/CD for a microservices project?

Use pipelines per service: continuous integration (build, unit tests, static analysis) producing a versioned artifact or container image pushed to a registry. On merge to main branch, run integration tests (or on PR). For CD, use automated deployments to staging (canary or blue-green strategies) and manual/automated promotion to production. Automate database migrations, use feature flags for safe rollouts, and ensure robust rollback procedures.

What is OpenShift and how does it differ from Kubernetes?

OpenShift is a platform built on Kubernetes that adds developer tooling, an integrated container registry, source-to-image (S2I) builds, a web console, and stricter security defaults (e.g., running containers as non-root by default). OpenShift includes additional abstractions (Routes, BuildConfig) and enterprise features. Think of OpenShift as an enterprise-ready distribution on top of Kubernetes.

How do you scale applications in OpenShift?

Scale by increasing replica counts in Deployment/DeploymentConfig, use Horizontal Pod Autoscaler (HPA) to scale based on CPU or custom metrics, configure resource requests/limits to inform scheduling, use Cluster Autoscaler to add nodes when needed, and partition workloads across namespaces or projects. Also use readiness/liveness probes to ensure healthy scaling.

Explain how Docker images are built and optimized.

Use multi-stage builds to separate build-time dependencies from runtime artifacts, minimize the number of layers by combining commands where sensible, use smaller base images (distroless or slim), leverage build cache ordering (put frequently changing steps later), and remove unnecessary files. Keep image sizes small to speed up pull/start times and improve security.

Testing (JUnit, Mockito)

What is the difference between unit testing and integration testing?

Unit tests isolate a single class or method and mock external dependencies to verify logic quickly and deterministically. Integration tests exercise multiple components together (e.g., repository + DB, or service + external API) to validate interactions and configuration. Integration tests are slower but catch integration and configuration issues unit tests cannot.

How do you mock external services in microservices tests?

Use Mockito to mock interfaces or Spring's @MockBean in slice tests. For HTTP services, WireMock or MockWebServer simulate endpoints with controllable responses. Use contract tests (Pact) to ensure compatibility with external teams, and use Testcontainers for full-stack integration tests against real infrastructure components.

Explain the use of Testcontainers in integration testing.

Testcontainers allows starting lightweight Docker containers (e.g., PostgreSQL, Kafka) within tests so you can run integration tests against real services. This increases test reliability and parity with production. Use reusable containers for speed, proper cleanup, and avoid relying on developer-local environments.

How do you test exception handling using Mockito?

Configure mocked methods to throw exceptions using when(...).thenThrow(...), then assert the system under test responds correctly (e.g., returns a fallback, wraps exception, or logs appropriately). Use JUnit5's assertThrows to verify that the expected exceptions are thrown.

```
when(mockedService.call()).thenThrow(new RuntimeException("fail"));
assertThrows(RuntimeException.class, () -> sut.doWork());
```

System Design & Leadership

Design a payment processing system with microservices, Kafka, and Redis.

Key components: API Gateway (auth, rate-limiting), Payment Service (validates requests, idempotency key), Order/Accounting Service (ledger), Notification Service, and external bank connectors. Use Kafka for asynchronous events—payment-initiated, payment-completed—and to decouple services. Use Redis for caching lookups and idempotency tokens for quick checks. For transaction integrity across services, consider a Saga pattern (choreography or orchestration) to coordinate distributed steps and compensating actions on failure. Ensure idempotency, retry logic, auditing, and strong observability (tracing, metrics, logs).

How would you ensure fault tolerance in a distributed system?

Design for failure: replicate services and state, use retries with exponential backoff, implement circuit breakers and bulkheads to isolate failures, use redundant data stores and backups, and prefer idempotent operations. Monitor health and set up alerting, autoscaling, and chaos testing to validate behavior. Use graceful degradation to maintain partial service if components fail.

Explain eventual consistency vs ACID transactions.

ACID transactions guarantee atomicity, consistency, isolation, and durability—useful when strong consistency is required. Eventual consistency accepts temporary inconsistencies but guarantees convergence; it scales well for distributed systems and high throughput (e.g., using asynchronous replication or event-driven updates). Choose based on business needs: financial ledger updates often need ACID or careful compensation, while user profile replication can tolerate eventual consistency.

How do you design APIs for scalability and maintainability?

Keep APIs stateless, support pagination for large datasets, use efficient filters and projections to limit payloads, version the API, provide clear error codes, and document contracts (OpenAPI). Use caching and throttling for heavy endpoints, and design idempotent write operations. Keep services small and cohesive; ensure consistent logging and tracing for observability.

As an AVP, how would you mentor junior developers on coding best practices?

Actions: perform constructive code reviews focused on teachable moments; run pairing sessions on complex tasks; create checklists for design and testing; introduce linters and static analysis; establish coding standards and architecture principles; encourage owning small features end-to-end and lead brown-bag sessions. Provide career guidance and set clear expectations aligned with business priorities.

How do you make technology trade-offs in architecture decisions?

Weigh factors: time-to-market, team familiarity, operational cost, performance needs, and long-term maintainability. Prototype risky choices (PoC) to measure trade-offs, quantify costs/benefits, and involve stakeholders in decisions. Prefer evolutionary architecture—start simple and refactor when necessary—while documenting assumptions and rollback paths.